

CSC 126 Computer Vision Midterm Project

Motion Detection and Object Tracking System Using Python and OpenCV

Recommended project title

Motion Detection and Object Tracking System Using Python and OpenCV

Difficulty

Beginner to Intermediate

Main technologies

- Python
- OpenCV
- NumPy
- VS Code
- Git/GitHub

Computer Vision topics used

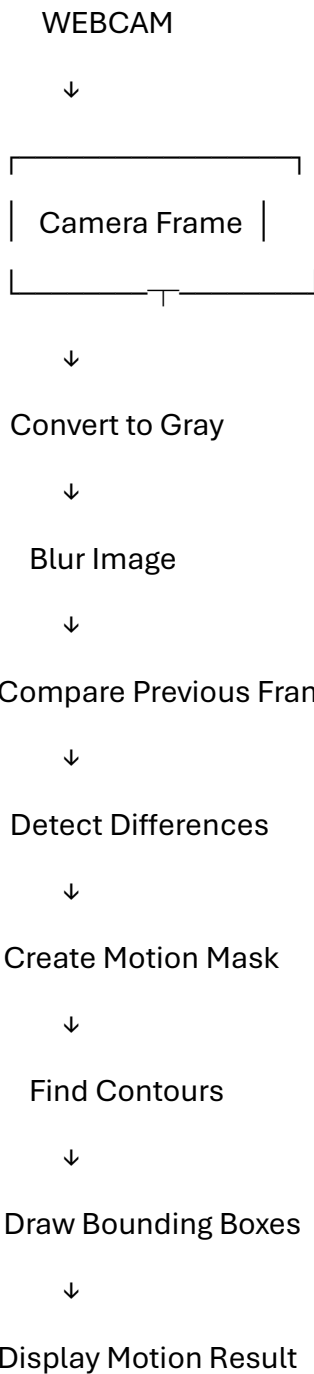
Laboratory Topic	Implementation
Image Operations	Grayscale conversion, Gaussian blur
Image Segmentation	Thresholding and binary motion mask
Motion and Object Tracking	Frame difference, contours, bounding boxes

PART 1 — Understand the Project

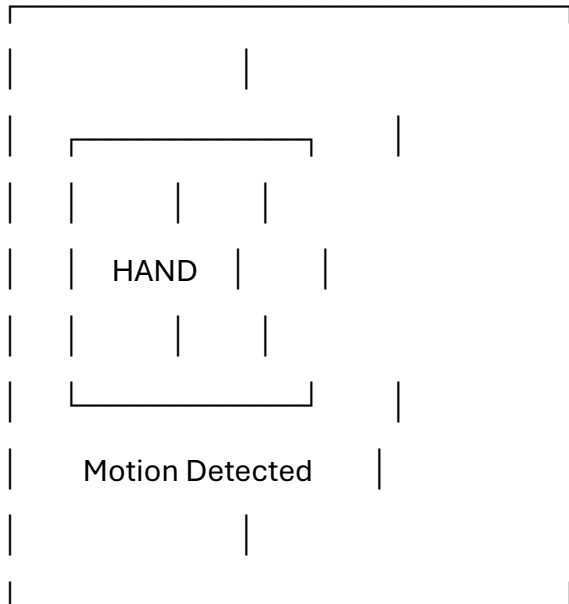
1.1 What are we building?

We are creating a Python desktop application that uses your **webcam** to detect movement.

For example:



When you move your hand:



The program will display something similar to:

Motion Detection System

Motion Detected

with a rectangle around the moving area.

PART 2 — Why This Project Fits CSC 126

Your project should not simply be a normal Python program.

It needs to demonstrate **Computer Vision concepts**.

Our project demonstrates three major topics.

Topic 1 — Image Operations

We process the camera image.

Grayscale

```
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
```

Instead of:

RGB/BGR

Red

Green

Blue

we convert the image into:

Black → White

or grayscale intensity.

Blur

```
gray = cv2.GaussianBlur(gray, (21, 21), 0)
```

This reduces small image noise.

Topic 2 — Image Segmentation

We need to separate:

Background

from:

Moving region

We do that using frame difference and thresholding.

For example:

```
difference = cv2.absdiff(previous_gray, gray)
```

Then:

```
_, mask = cv2.threshold( difference, 25, 255, cv2.THRESH_BINARY)
```

The result is a binary image:

BLACK = No significant change

WHITE = Motion detected

Topic 3 — Motion and Object Tracking

We find the white regions.

```
contours, _ = cv2.findContours( mask, cv2.RETR_EXTERNAL,  
cv2.CHAIN_APPROX_SIMPLE)
```

Then we calculate their area:

```
area = cv2.contourArea(contour)
```

Small areas are ignored.

Large areas are considered possible motion.

Then:

```
x, y, w, h = cv2.boundingRect(contour)
```

creates a bounding box.

Finally:

```
cv2.rectangle(...)
```

draws the box.

PART 3 — Create the Project Folder

Create a folder anywhere convenient.

For example:

A:\CSC 126\MotionDetectionProject

Your final project should look like:

MotionDetectionProject/

```
|
|
|— motion_detection.py
|— README.md
|— requirements.txt
|
|— samples/
|   |— sample1.jpg
|   └— sample2.jpg
|
|— screenshots/
|   |— setup.png
|   |— motion_detected.png
|   |— motion_mask.png
|   └— testing.png
|
└— documentation/
    └— CSC126_Midterm_Documentation.pdf
```

You don't have to create everything immediately.

Start with:

MotionDetectionProject/

PART 4 — Open the Project in VS Code

Open VS Code.

Select:

File → Open Folder

Choose:

MotionDetectionProject

Then open:

Terminal → New Terminal

You should see something like:

```
PS A:\CSC 126\MotionDetectionProject>
```

PART 5 — Check Python

Run:

```
python --version
```

You should see something similar to:

```
Python 3.x.x
```

For example:

```
Python 3.12.10
```

If Python works, continue.

PART 6 — Create a Virtual Environment

Run:

```
python -m venv venv
```

This creates:

```
MotionDetectionProject/
```

```
|
```

```
└─ venv/
```

The virtual environment keeps your project packages separate from your other Python projects.

PART 7 — Activate the Virtual Environment

On Windows PowerShell:

```
.\env\Scripts\Activate.ps1
```

If successful, your terminal should show:

```
(venv) PS A:\CSC 126\MotionDetectionProject>
```

The important part is:

```
(venv)
```

If PowerShell blocks activation

You might get:

running scripts is disabled

Don't panic.

You can use Command Prompt instead.

Open CMD and navigate to the project:

```
cd /d "A:\CSC 126\MotionDetectionProject"
```

Then:

```
venv\Scripts\activate.bat
```

You should get:

```
(venv)
```

PART 8 — Install OpenCV and NumPy

Run:

```
python -m pip install opencv-python numpy
```

Wait until installation finishes.

Then verify OpenCV:

```
python -c "import cv2; print(cv2.__version__)"
```

You should get something like:

4.12.0

Also test NumPy:

```
python -c "import numpy; print(numpy.__version__)"
```

PART 9 — Create requirements.txt

Create:

requirements.txt

Put:

opencv-python

numpy

This file tells another person what packages are required.

Later, someone can install everything using:

```
pip install -r requirements.txt
```

PART 10 — Create the Python Program

Create:

motion_detection.py

Now we'll build the program properly.

Complete beginner-friendly code

Use this version rather than the incomplete code in your previous message:

```
import cv2
if motion_detected:    status = "Motion Detected"    status_color = (0, 0,
255) else:    status = "No Motion"    status_color = (255, 255, 255)
cv2.putText(frame, status, (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 0.8, status_color,
2 ) # ===== # DISPLAY WINDOWS #
===== cv2.imshow( "Motion Detection System",
```

```
frame ) cv2.imshow( "Motion Mask", mask ) #
===== # SAVE CURRENT FRAME #
===== previous_gray = gray #
===== # EXIT #
===== if cv2.waitKey(1) & 0xFF == ord("q"): break#
===== # CLEANUP#
=====cap.release()cv2.destroyAllWindows()print("Motion
Detection System Closed")
```

PART 11 — Understand the Code

Don't just copy the code.

You need to understand it because your instructor may ask questions during the **5-minute Q&A**.

Let's break it down.

11.1 Import OpenCV

import cv2

cv2 is the Python library for OpenCV.

It allows us to:

- access webcam
 - process images
 - convert colors
 - blur images
 - detect contours
 - draw rectangles
 - display images
-

11.2 Camera index

`CAMERA_INDEX = 0`

This tells OpenCV which camera to use.

Usually:

0 = built-in/default webcam

1 = second camera

2 = third camera

For your laptop, start with:

`CAMERA_INDEX = 0`

11.3 Minimum area

`MIN_AREA = 1000`

This prevents tiny changes from being detected as motion.

For example:

small noise

small shadow

small pixel changes

can be ignored.

If you set:

`MIN_AREA = 100`

the system becomes more sensitive.

If you set:

`MIN_AREA = 3000`

the system becomes less sensitive.

11.4 Start the webcam

```
cap = cv2.VideoCapture(CAMERA_INDEX)
```

This opens the webcam.

Think of:

cap

as the connection to your camera.

11.5 Read the camera

```
ret, frame = cap.read()
```

This gets one frame.

For example:

Frame 1

Frame 2

Frame 3

Frame 4

Frame 5

...

A webcam is basically giving us many images very quickly.

11.6 Convert to grayscale

```
gray = cv2.cvtColor( frame, cv2.COLOR_BGR2GRAY)
```

Why?

Because motion detection does not necessarily need full color information.

Instead of processing:

Blue

Green

Red

we process:

Brightness

This also makes the processing simpler.

11.7 Gaussian blur

```
gray = cv2.GaussianBlur( gray, (21, 21), 0)
```

This removes small noise.

Imagine your image contains tiny random changes:

.....

Blurring makes the image smoother.

This helps reduce false motion detection.

11.8 Store previous frame

```
previous_gray = None
```

At the beginning, there is no previous frame.

After reading the first frame:

```
previous_gray = gray
```

Now we have:

Previous frame

↓

Current frame

11.9 Compare frames

This is one of the most important parts:

```
difference = cv2.absdiff( previous_gray, gray)
```

Suppose:

Previous frame

Person on left

Current frame

Person on right

There will be a difference between the two frames.

OpenCV calculates:

Current Frame

-

Previous Frame

=

Difference

The difference represents areas that changed.

11.10 Threshold

```
_, mask = cv2.threshold( difference, THRESHOLD_VALUE, 255, cv2.THRESH_BINARY)
```

This converts the difference into black and white.

Basically:

Small difference → BLACK

Large difference → WHITE

Therefore:

BLACK

=

No significant motion

WHITE

=

Possible motion

11.11 Dilation

```
mask = cv2.dilate( mask, None, iterations=2)
```

Dilation expands white areas.

This helps connect nearby detected regions.

Without it, motion may look like:

white white

white

After dilation:



It becomes easier to detect as one region.

11.12 Find contours

```
contours, _ = cv2.findContours( mask, cv2.RETR_EXTERNAL,  
cv2.CHAIN_APPROX_SIMPLE)
```

Contours represent boundaries of detected regions.

For example:



OpenCV can identify its boundary.

11.13 Calculate area

`area = cv2.contourArea(contour)`

Suppose:

Area = 50

That's probably noise.

But:

Area = 20,000

could be a moving person or object.

That's why we use:

if area < MIN_AREA: continue

11.14 Bounding rectangle

`x, y, w, h = cv2.boundingRect(contour)`

This gives:

x = horizontal position

y = vertical position

w = width

h = height

Then:

`cv2.rectangle(...)`

draws the box.

11.15 Display label

`cv2.putText( frame, "Motion Detected", ...)`

This places text on the video.

11.16 Display the result

```
cv2.imshow( "Motion Detection System", frame)
```

This shows the camera with bounding boxes.

And:

```
cv2.imshow( "Motion Mask", mask)
```

shows the segmentation result.

PART 12 — Run the Program

Make sure your virtual environment is active:

```
(venv)
```

Then:

```
python motion_detection.py
```

You should see:

Motion Detection System Started

Move something in front of the camera.

Press Q to quit.

Two windows should open.

Window 1

Motion Detection System

Shows your camera.

Window 2

Motion Mask

Shows the detected motion as white regions.

PART 13 — First Test

Don't immediately move.

Sit still for a few seconds.

You should see:

No Motion

Then move your hand.

You should see:

Motion Detected

and a green bounding box.

PART 14 — Test Plan

For your project documentation, perform several tests.

Test 1 — No movement

	Test Action	Expected Result
--	--------------------	------------------------

1	Sit still	No Motion
---	-----------	-----------

Expected:

No Motion

Test 2 — Move hand

	Test Action	Expected Result
--	--------------------	------------------------

2	Move hand	Motion Detected
---	-----------	-----------------

Expected:

Green bounding box

Motion Detected

Test 3 — Move object

Put a bottle or other object in front of the camera.

Move it.

Expected:

Motion Detected

Test 4 — Multiple moving objects

Move two objects.

Expected:

Box 1

Box 2

The system should detect multiple contours.

Test 5 — Lighting change

Turn a light on/off.

You may see:

Motion Detected

This is an important limitation.

The system detects **pixel changes**, not necessarily actual objects.

PART 15 — Important Limitation

This is something you should explain during your presentation.

Our system is called:

Motion Detection and Object Tracking

But technically, this beginner version is primarily **motion detection with bounding regions**.

It does not permanently identify:

Person A

Person B

Car 1

Car 2

It simply detects areas that changed between frames.

Therefore:

It can detect:

- moving hand
- moving person
- moving object
- some moving vehicles
- moving shadows

It can also falsely detect:

- camera movement
- lighting changes
- shadows
- sudden brightness changes

This is not necessarily a problem for a beginner project.

In fact, mentioning limitations makes your presentation stronger because you understand what your system actually does.

PART 16 — Improve the Project for Better Presentation

I recommend adding a few features.

Feature 1 — FPS

You can display:

FPS: 30

This shows performance.

Feature 2 — Motion count

You can display:

Objects Detected: 2

This makes the application look more complete.

Feature 3 — Timestamp

You can display:

2026-10-08 15:30:22

This makes it look more like a monitoring system.

Feature 4 — Instructions

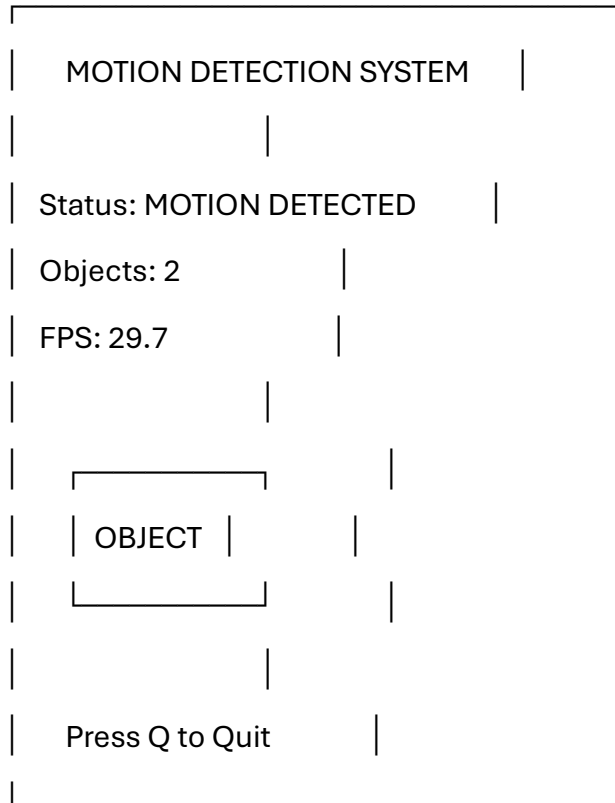
Display:

Press Q to Quit

PART 17 — Improved Version

After the basic version works, we can upgrade it.

For example:



But do not add too many features before your basic system works.

First:

Camera

↓

Grayscale

↓

Blur

↓

Difference

↓

Threshold

↓

Contours

↓

Bounding boxes

Then improvements.

PART 18 — Suggested Final Folder

Your final project should look like:

MotionDetectionProject/

|

|— motion_detection.py

|

|— requirements.txt

|

|— README.md

|

|— samples/

|

| |— sample1.jpg

|

| |— sample2.jpg

|

| |— sample3.jpg

|

|— screenshots/

|

| |— 01_no_motion.png

|

| |— 02_motion_detected.png

```
| |— 03_motion_mask.png
| |— 04_multiple_objects.png
| |— 05_testing.png
|
|— documentation/
    |— CSC126_Midterm_Documentation.pdf
```

PART 19 — README.md

Your README should explain the project.

You can use this structure:

```
# Motion Detection and Object Tracking System
```

```
## CSC 126 Computer Vision Midterm Project
```

```
### Description
```

The Motion Detection and Object Tracking System is a Python desktop application that uses OpenCV to detect movement through a webcam.

The system compares consecutive video frames and identifies areas where significant changes occur. Detected moving regions are displayed using bounding boxes.

```
### Objectives
```

- Detect motion using a webcam.
- Apply image processing techniques.

- Segment moving regions from the background.
- Detect contours.
- Display bounding boxes around moving regions.

Computer Vision Topics Used

1. Image Operations
2. Image Segmentation
3. Motion and Object Tracking

Technologies

- Python
- OpenCV
- NumPy

Requirements

- Python 3.x
- Webcam
- Windows/macOS/Linux
- OpenCV
- NumPy

Installation

Clone the repository:

```
```bash
```

```
git clone YOUR_GITHUB_REPOSITORY_URL
```

Navigate to the project:

```
cd MotionDetectionProject
```

Create a virtual environment:

```
python -m venv venv
```

Activate the environment.

Windows PowerShell:

```
.\venv\Scripts\Activate.ps1
```

Install dependencies:

```
pip install -r requirements.txt
```

## Run

```
python motion_detection.py
```

Move an object in front of the webcam.

Press Q to quit.

## How It Works

The system follows these steps:

1. Capture webcam frame.
2. Convert frame to grayscale.
3. Apply Gaussian blur.
4. Compare current frame with previous frame.
5. Apply thresholding.
6. Find contours.
7. Filter small regions.

8. Draw bounding boxes.
9. Display the result.

### **Limitations**

The system detects changes between frames rather than identifying specific people or objects.

Changes in lighting, camera movement, and shadows may cause false motion detection.

### **Future Improvements**

Possible future improvements include:

- Object tracking
- Background subtraction
- Person detection
- Motion recording
- Motion alerts
- Database logging
- Web dashboard

### **Author**

Your Name

CSC 126 - Computer Vision

---

### **# PART 20 — GitHub Setup**

Once your program works, create a GitHub repository.

For example:

```text

CSC126-Motion-Detection

Your GitHub repository could look like:

CSC126-Motion-Detection

|

├─ motion_detection.py

├─ requirements.txt

├─ README.md

├─ samples/

├─ screenshots/

└─ documentation/

PART 21 — Git Commands

Inside your project folder:

git init

Then:

git add .

Commit:

git commit -m "Initial motion detection system"

Connect GitHub:

git remote add origin YOUR_REPOSITORY_URL

Then:

git branch -M main

Finally:

git push -u origin main

PART 22 — Very Important: Don't Upload venv

Do **not** upload:

venv/

The virtual environment can be very large.

Create:

.gitignore

Put:

venv/

__pycache__/

*.pyc

Then Git will ignore those files.

Your repository should contain:

motion_detection.py

README.md

requirements.txt

samples/

screenshots/

documentation/

but not:

venv/

PART 23 — Documentation PDF

Your instructor requires a **2–4 page PDF documentation**.

I recommend this structure.

Page 1 — Introduction

Title

Motion Detection and Object Tracking System Using Python and OpenCV

Introduction

Explain:

- What the project is
- What problem it solves
- Why motion detection is useful

Problem

Example:

Monitoring an area manually can be difficult because a person cannot continuously watch a camera feed. The proposed system provides a simple computer vision solution that detects movement automatically.

Objective

Example:

The objective of this project is to develop a Python-based computer vision application that detects moving regions from a webcam feed and displays bounding boxes around detected motion.

Page 2 — Methodology

Explain the processing pipeline:

Webcam

↓

Frame Capture

↓

Grayscale

↓

Gaussian Blur

↓

Frame Difference

↓

Threshold

↓

Binary Mask

↓

Contour Detection

↓

Bounding Box

↓

Display Result

Explain each step.

Page 3 — Results

Add screenshots.

For example:

Figure 1 — No Motion

Screenshot showing:

No Motion

Figure 2 — Motion Detected

Screenshot showing:

Motion Detected

with bounding box.

Figure 3 — Motion Mask

Screenshot of the binary mask.

Then discuss your testing.

Example:

| Test | Result |
|------------------|-----------------|
| No movement | Passed |
| Hand movement | Passed |
| Object movement | Passed |
| Multiple objects | Passed |
| Lighting change | Motion detected |

Page 4 — Conclusion and Limitations

Conclusion

Explain that the system successfully demonstrates three computer vision concepts:

1. Image Operations
2. Image Segmentation
3. Motion Detection/Object Tracking

Limitations

Mention:

- lighting changes
- camera movement
- shadows
- no object identity
- basic frame comparison

Future Improvements

Mention:

- object tracking

- person detection
 - motion recording
 - alerts
 - background subtraction
-

PART 24 — Screenshots You Should Take

You should collect screenshots while working.

At minimum:

Screenshot 1

VS Code project:

motion_detection.py

requirements.txt

README.md

Screenshot 2

Program running with:

No Motion

Screenshot 3

Program running with:

Motion Detected

and bounding box.

Screenshot 4

Motion mask:

black background

white moving object

Screenshot 5

Multiple moving objects.

Screenshot 6

Terminal showing successful execution.

For example:

Motion Detection System Started

Move something in front of the camera.

Press Q to quit.

PART 25 — How to Demonstrate It

You have around **15 minutes**.

Don't spend 10 minutes explaining code.

Use this structure.

0–2 minutes — Introduction

Say:

Good day everyone. Our project is called Motion Detection and Object Tracking System Using Python and OpenCV. The purpose of this project is to detect movement using a webcam and highlight moving regions using bounding boxes.

2–4 minutes — Problem

Explain:

Manual monitoring requires continuous human attention. Our system provides a basic computer vision solution that automatically detects changes in a camera feed.

4–6 minutes — Computer Vision Concepts

Explain your three topics.

Image Operations

First, we convert the frame to grayscale and apply Gaussian blur to reduce noise.

Image Segmentation

Next, we compare consecutive frames and use thresholding to create a binary mask that separates changed regions from the background.

Motion Detection

Finally, we detect contours and draw bounding boxes around significant moving regions.

6–11 minutes — Live Demonstration

Run:

```
python motion_detection.py
```

Then demonstrate:

Demo 1

Stay still.

No Motion

Demo 2

Move your hand.

Motion Detected

Demo 3

Move an object.

Motion Detected

Demo 4

Move multiple objects.

Show multiple bounding boxes.

11–13 minutes — Explain Code

Show:

```
gray = cv2.cvtColor(...)
```

Explain grayscale.

Then:

```
difference = cv2.absdiff(...)
```

Explain frame comparison.

Then:

```
cv2.findContours(...)
```

Explain motion region detection.

Then:

```
cv2.boundingRect(...)
```

Explain bounding box.

13–15 minutes — Results and Conclusion

Say:

Based on our testing, the system successfully detected movement from the webcam and displayed bounding boxes around significant moving regions. However, the system may also detect lighting changes or shadows because it detects pixel differences rather than understanding the identity of objects.

That's a strong conclusion because you're acknowledging the limitation.

PART 26 — Possible Q&A Questions

Your instructor may ask these.

Question 1

What is OpenCV?

Answer:

OpenCV is an open-source computer vision library used for processing images and videos.

Question 2

Why did you convert the image to grayscale?

Answer:

We converted the image to grayscale because motion detection mainly needs brightness information, and grayscale reduces the amount of data that needs to be processed.

Question 3

What is Gaussian blur?

Answer:

Gaussian blur reduces small image noise and makes motion detection more stable.

Question 4

What is image segmentation?

Answer:

Image segmentation separates important regions of an image from the background. In our project, we use a binary mask to separate areas that changed from areas that did not change.

Question 5

How does your system detect motion?

Answer:

It compares the current frame with the previous frame. Significant differences between the frames are treated as possible motion.

Question 6

What is a contour?

Answer:

A contour is a boundary around a connected region in an image. We use contours to identify the detected moving regions.

Question 7

What is a bounding box?

Answer:

A bounding box is a rectangle surrounding a detected region. In our project, it shows where motion was detected.

Question 8

What happens if the camera moves?

Answer:

The system may detect the entire scene as motion because it relies on differences between consecutive frames.

Question 9

Can your system recognize a person?

Answer:

No. The current version detects motion regions but does not identify or recognize specific people.

Question 10

Why do you use MIN_AREA?

Answer:

MIN_AREA filters out very small regions caused by noise or insignificant pixel changes.

Question 11

Why use thresholding?

Answer:

Thresholding converts the frame difference into a binary image, making it easier to separate significant changes from insignificant changes.

Question 12

What happens if you increase the threshold?

Answer:

The system becomes less sensitive to small changes, so only larger differences are more likely to be detected.

PART 27 — Common Problems and Fixes

Problem 1 — No module named cv2

You may see:

ModuleNotFoundError: No module named 'cv2'

Run:

```
python -m pip install opencv-python
```

Problem 2 — No module named numpy

Run:

```
python -m pip install numpy
```

Problem 3 — Camera doesn't open

If:

```
CAMERA_INDEX = 0
```

doesn't work, try:

```
CAMERA_INDEX = 1
```

Also close:

- Zoom

- Google Meet
 - Microsoft Teams
 - Camera app
 - other applications using your webcam
-

Problem 4 — Window freezes

Make sure this exists:

```
cv2.waitKey(1)
```

Without waitKey(), OpenCV windows may not update properly.

Problem 5 — Pressing Q doesn't work

Click the OpenCV window first.

Then press:

Q

Problem 6 — Too much motion detection

Increase:

```
MIN_AREA = 1000
```

to:

```
MIN_AREA = 2000
```

or:

```
MIN_AREA = 3000
```

Problem 7 — Motion is not detected

Try lowering:

```
MIN_AREA
```

For example:

MIN_AREA = 500

You can also lower:

THRESHOLD_VALUE = 25

to:

THRESHOLD_VALUE = 15

PART 28 — Development Order

This is very important.

Don't try to create the entire project in one sitting.

Follow this order:

Stage 1

Install Python.

↓

Stage 2

Create virtual environment.

↓

Stage 3

Install OpenCV.

↓

Stage 4

Test webcam.

↓

Stage 5

Test grayscale.

↓

Stage 6

Test frame difference.



Stage 7

Test threshold.



Stage 8

Test contours.



Stage 9

Add bounding boxes.



Stage 10

Add status text.



Stage 11

Test the complete system.



Stage 12

Take screenshots.



Stage 13

Create README.



Stage 14

Create documentation PDF.

↓

Stage 15

Push to GitHub.

↓

Stage 16

Practice presentation.

PART 29 — Final Submission Checklist

Before submitting, check everything.

Program

- Python program works
- Webcam opens
- Motion detection works
- Bounding boxes work
- Motion mask works
- Q exits the application

Computer Vision

- Image Operations implemented
- Image Segmentation implemented
- Motion/Object Detection implemented

Files

- motion_detection.py
- requirements.txt
- README.md
- sample inputs
- screenshots

- PDF documentation

GitHub

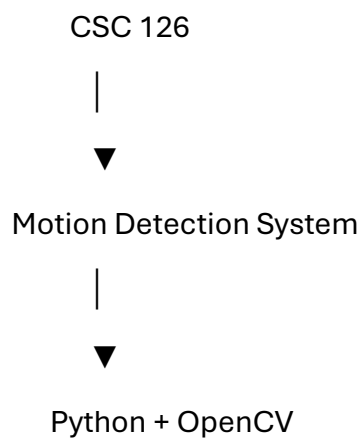
- Repository created
- Code pushed
- README uploaded
- .gitignore added
- venv/ NOT uploaded
- Repository link works

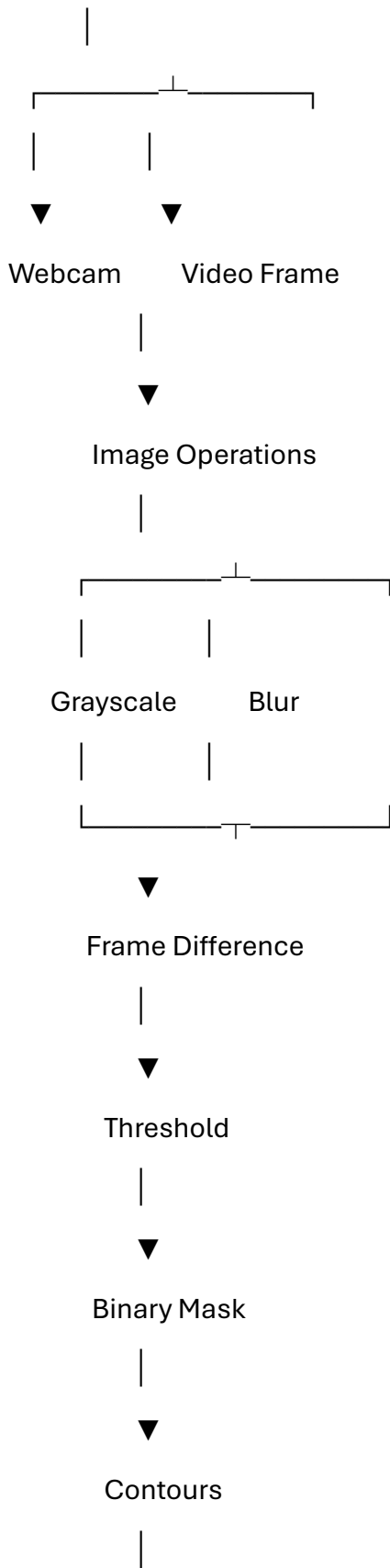
Presentation

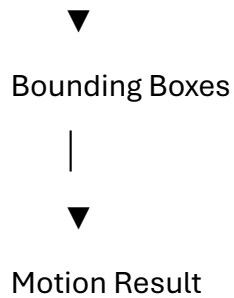
- Project introduction prepared
- Problem explained
- Methodology explained
- Live demo tested
- Results prepared
- Limitations understood
- Q&A practiced

PART 30 — Your Final Project Architecture

At the end, your project can be summarized like this:







The most important thing

Don't add advanced features yet.

First make this work:

Webcam

↓

Grayscale

↓

Blur

↓

Frame Difference

↓

Threshold

↓

Contours

↓

Bounding Box

↓

Motion Detected

Once that works reliably, we can make it look more like a polished **desktop application** with a proper GUI, buttons, status indicators, object count, FPS, screenshots, and a cleaner presentation.

Recommended next development sequence

Step 1: Get the basic motion_detection.py above running.

Step 2: Test your webcam and make sure you can get the green bounding box.

Step 3: Upgrade it into a more polished **CSC 126 desktop application**.

Step 4: Create the README and 2–4 page documentation.

Step 5: Prepare the GitHub repository and presentation/Q&A.